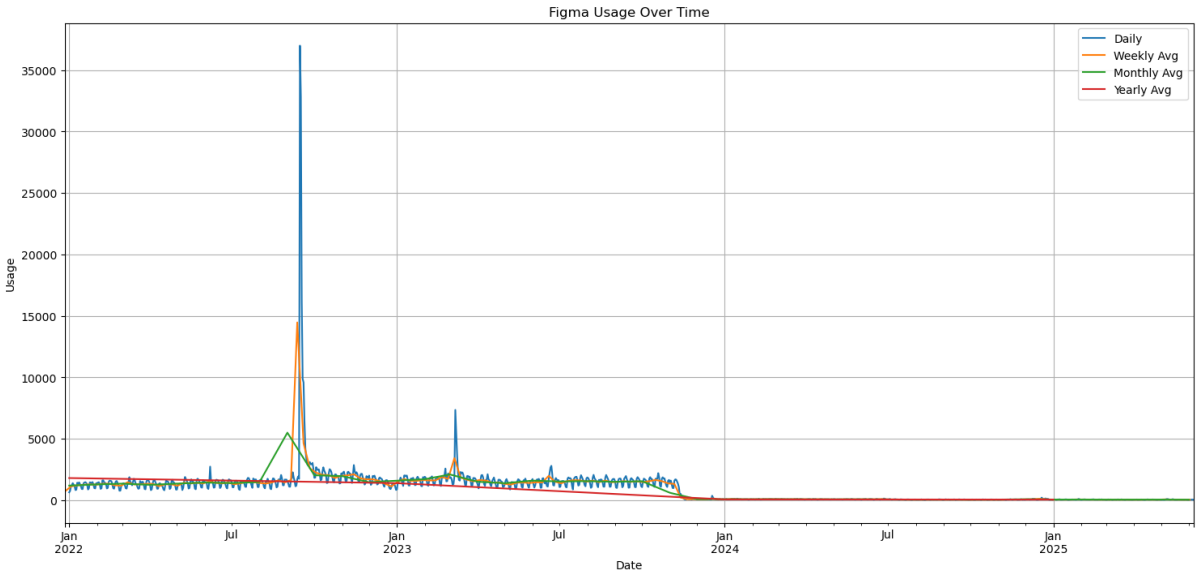
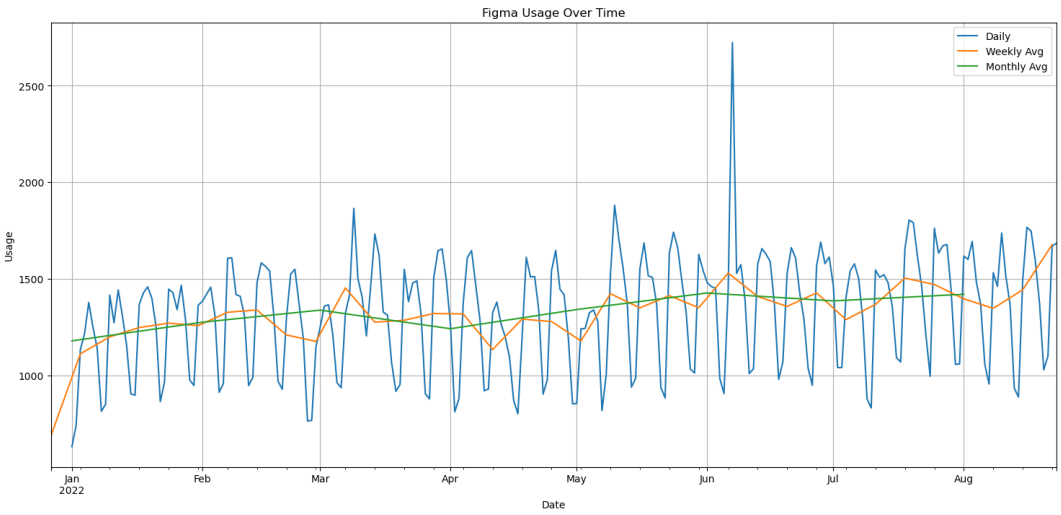
1. Initial plotting and observation:



1. Overall Pageviews (2022–2025)

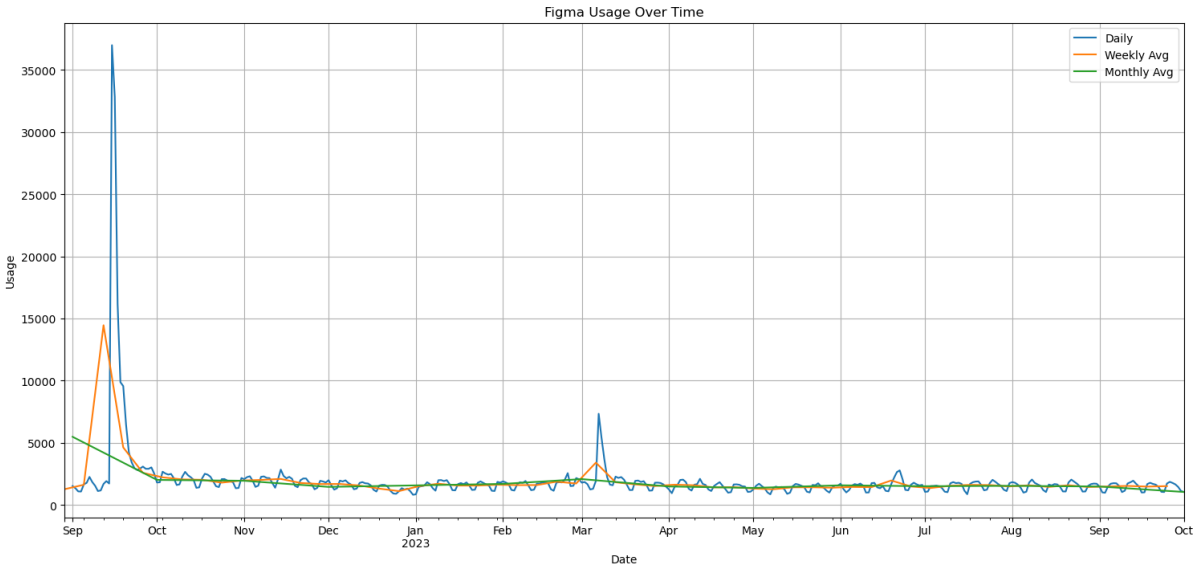
The overall trend shows a rise in Figma Wikipedia pageviews during early 2022, peaking dramatically in September 2022 (likely due to the Adobe acquisition news), followed by a steady decline and flattening through 2024–2025. Strong weekly seasonality is evident in early periods but fades post-2023. The mean increases initially, then drops significantly, while variance spikes during the anomaly and contracts later. Autocorrelation is high before the dip, especially at weekly lags, but diminishes afterward. Major outliers dominate the series, particularly the September 2022 spike and a few smaller events.



2. Before the Big Spike (~Jan–Aug 2022)

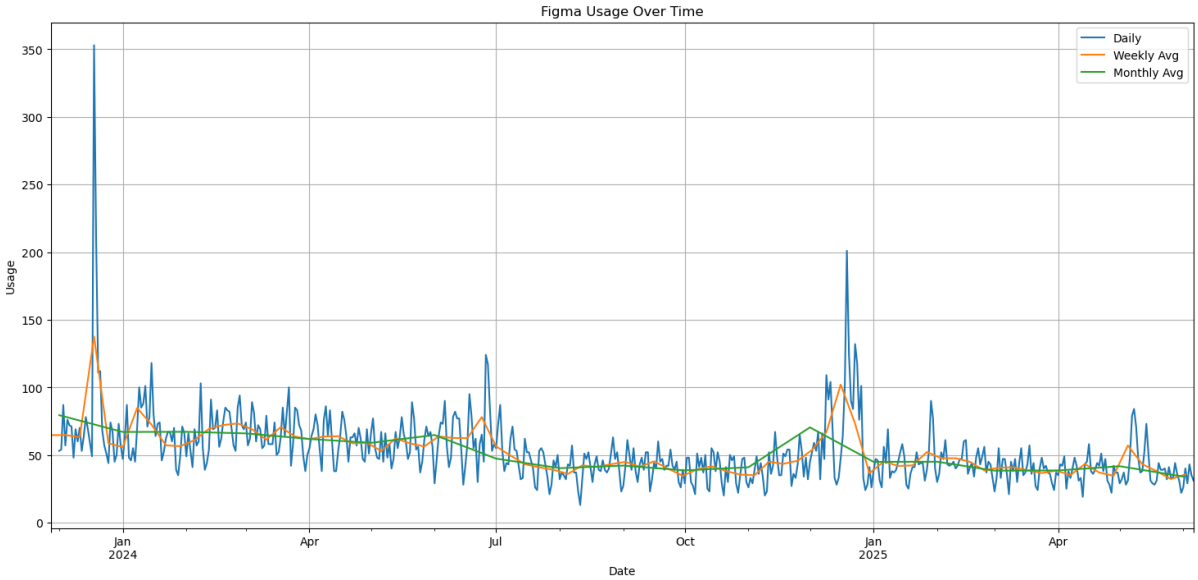
This period shows a clear upward trend with strong and regular weekly seasonality. The mean steadily rises and the variance remains moderate, reflecting consistent user

interest and growth. Residual noise is minimal, and autocorrelation is high, especially weekly. Aside from a minor spike in June 2022, the data is smooth and structured, making it ideal for forecasting models that account for trend and seasonality.



3. Post-Spike to Dip (~Sept 2022–end 2023)

Following the major spike, pageviews exhibit a downward trend with gradually stabilizing behavior. Seasonality weakens but remains present, while residual noise increases due to intermittent spikes in March and July 2023. The mean and variance decrease compared to the spike period, and autocorrelation is less pronounced but still visible weekly. This phase reflects decaying public interest with some noise from external events.



4. Post Dip (2024–mid 2025)

Pageviews drop to a low baseline with a flat or slightly declining trend. Seasonality disappears almost entirely, and the data is dominated by low-magnitude noise. Mean

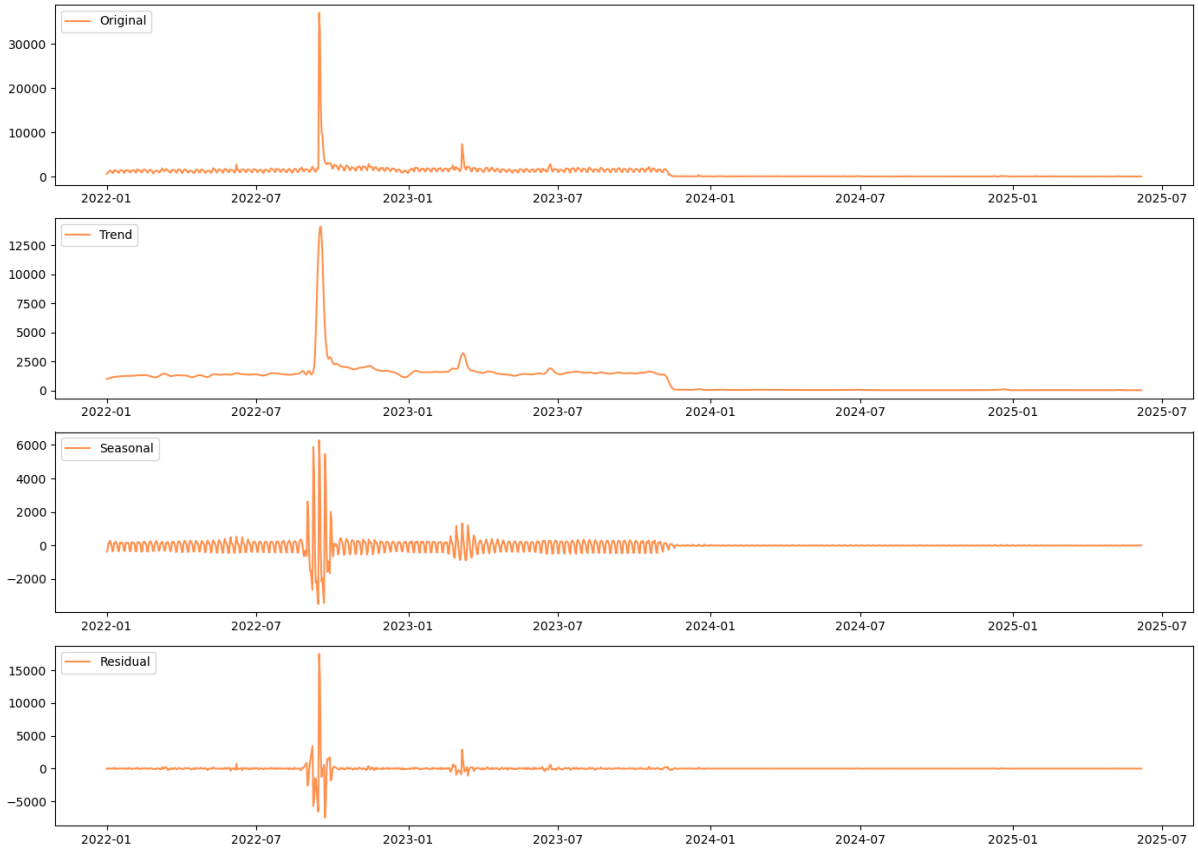
usage is minimal, variance is very low, and autocorrelation is virtually absent. A couple of mild spikes appear, but the overall pattern suggests either diminished user interest or potential issues in data collection or completeness.

2. Performing decomposition

Here we will perform additive decomposition to better capture the trend, seasonality and noise.

Preferred model-> STL decomposition using Local Estimated Scatterplot smoothening to handle outliers better and provides more flexibility to adjust smoothing parameters for the trend and seasonal components

1. **Overall Pageviews (2022–2025)**



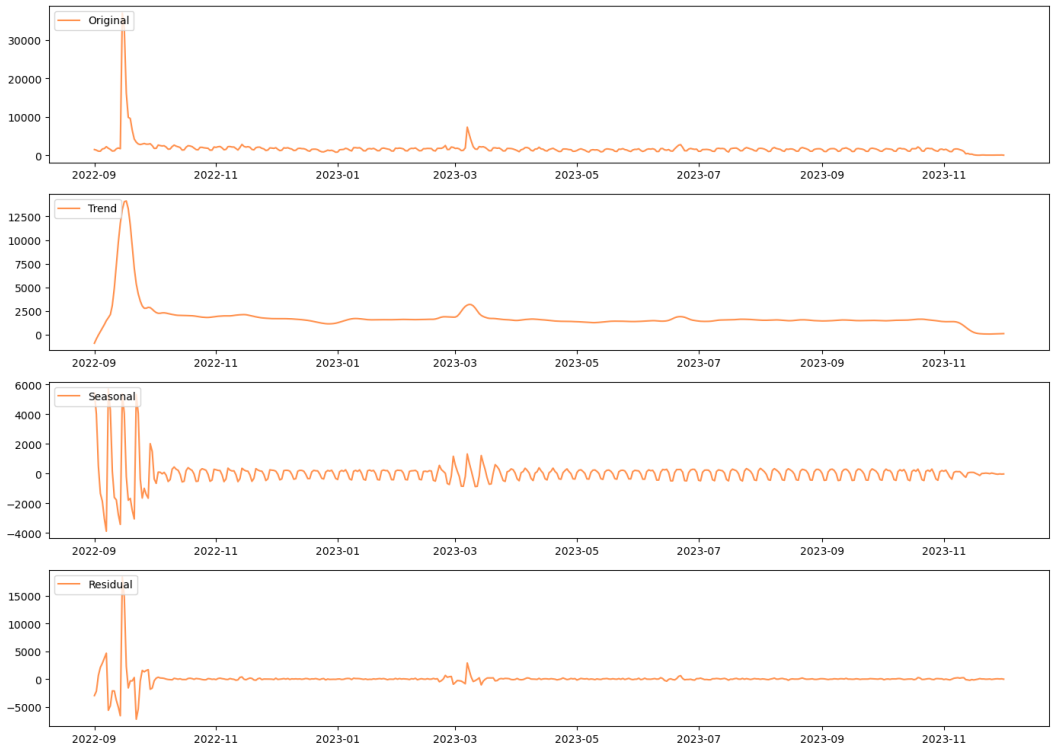
Using a 7-day period for STL decomposition effectively captures the dominant weekly seasonality observed in the Figma Wikipedia pageviews. The trend component clearly highlights long-term shifts, including a major spike around September 2022 and a gradual decline thereafter. The seasonal component reflects consistent weekly user behavior during high-traffic periods, aligning with expected weekday/weekend usage patterns. The residual component isolates anomalies and irregularities, such as sudden spikes due to external events. Overall, the 7-day period is ideal for this dataset, as it

balances short-term fluctuation with interpretability and preserves meaningful seasonality without over-smoothing.

2. **Before the Big Spike (~Jan–Aug 2022)**

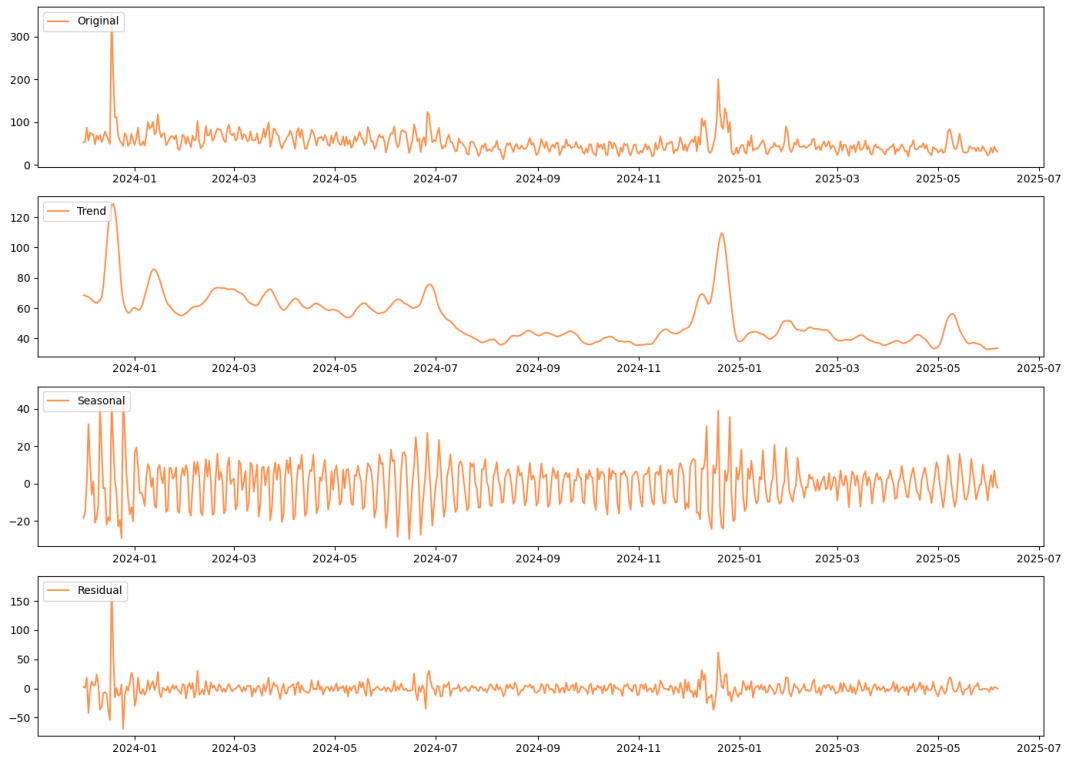
This period shows a steadily increasing trend, reflecting growing interest in Figma. Weekly seasonality is strong and consistent, with regular peaks and troughs aligning with weekday usage. Residual noise is low and stable, except for a small spike in June that stands out as a mild anomaly. The signal is clean, with high autocorrelation and a gradually rising mean, making this a well-behaved and forecast-friendly segment.

3. **Post-Spike to Dip (~Sept 2022–end 2023)**



Immediately after the massive September spike, the trend sharply declines and then flattens out. Seasonality weakens and becomes less structured over time. Residuals capture the shock of the spike and later irregularities, such as minor bumps in early 2023. The mean drops significantly while variance briefly remains high before stabilizing. This phase reflects the decay of peak attention and growing noise, with some loss of predictable structure.

4. **Post Dip (2024–mid 2025)**



Following the dip, the trend remains flat and low with a few brief upward bumps, likely reflecting isolated events. Seasonality becomes subtle but re-emerges in patches, particularly around early 2025. Residuals are small and mostly stable, though minor outliers remain visible. Overall, the mean is very low, variance minimal, and autocorrelation weak — indicating a quiet, low-traffic period with occasional disruptions.

3. Stationary checks for classical models:

To apply classical time series models like ARIMA, we must verify whether the data is stationary — that is, whether its statistical properties (mean, variance, autocorrelation) remain constant over time.
Based on visual inspection of the original and STL-decomposed components, the data exhibits a **non-stationary behavior**. The trend is not constant — it rises gradually during early 2022, spikes sharply in September 2022, then declines and flattens. Seasonal strength also varies, being prominent initially and fading later. The variance is not strictly stable due to these shifts and large residual spikes. Autocorrelation is strong early on but diminishes post-2023. These signs indicate non-stationarity due to changing mean and variance, and inconsistent seasonality over time.
To confirm this observation, we perform two statistical tests:

ADF (Augmented Dickey-Fuller) Test – Null hypothesis: the data is non-stationary.

KPSS (Kwiatkowski–Phillips–Schmidt–Shin) Test – Null hypothesis: the data is stationary.

1. We first perform Ljung-Box test:

   For this, we use Ljung-Box test

```
]: from statsmodels.stats.diagnostic import acorr_ljungbox
   lb_test_white_noise = acorr_ljungbox(data, lags=[10], return_df=True)
   print(lb_test_white_noise)
```

```
        lb_stat  lb_pvalue
10  2129.055692        0.0
```

   Conclusion-> The series is not white noise — it contains significant autocorrelation and structure.

2. Then we perform AD-Fuller test:

```
ADF Test Results:
ADF Statistic: -5.402
p-value: 0.0
Critical Values:
   1%: -3.436
   5%: -2.864
   10%: -2.568
```

   Conclusion-> We reject Null hypothesis. This makes the model stationary.

3. Then to validate, and check for mean or trend variation over time, we check KPSS test:

```
KPSS Test Results:
KPSS Statistic: 0.27
p-value: 0.01
Critical Values:
   10%: 0.119
   5%: 0.146
   2.5%: 0.176
   1%: 0.216
```

For ct-> Trend line variation with time.

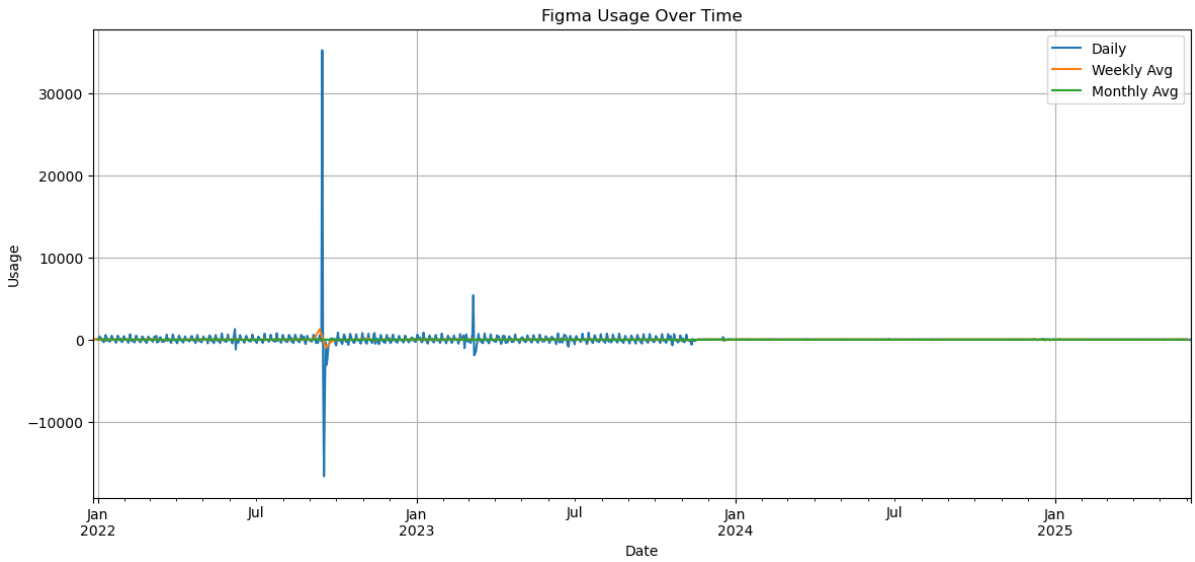
```
KPSS Test Results:
KPSS Statistic: 3.26
p-value: 0.01
Critical Values:
   10%: 0.347
   5%: 0.463
   2.5%: 0.574
   1%: 0.739
```

For c->the mean changes over time. This aligns with the presence of trend, as seen in STL decomposition.

Outcome-> KPSS states that the data trend-wise and mean wise vary over time and is not stationary.

4. Making the model stationary for classical models

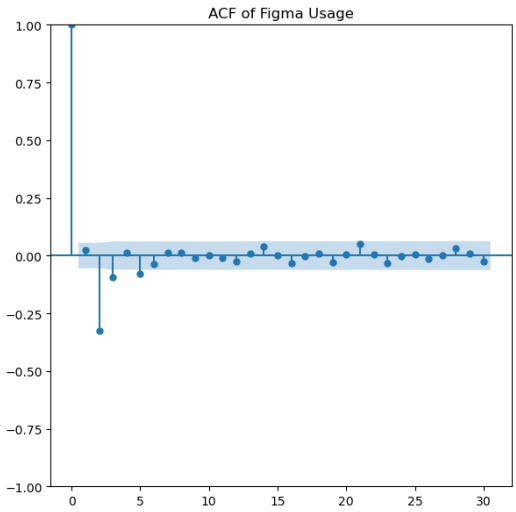
Therefore to tackle this issue our approach-> We will apply first-order differencing.

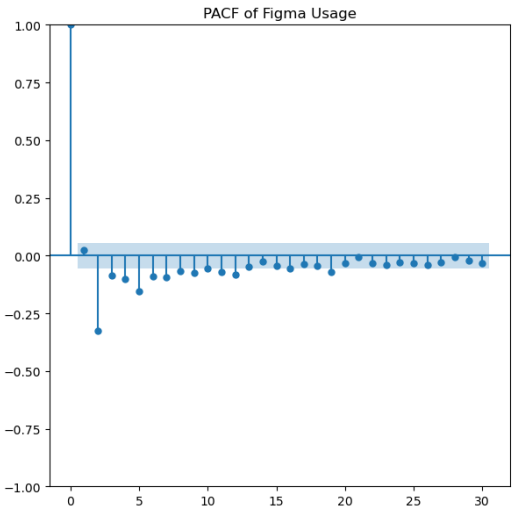


Conclusion-> After first-order differencing, the series shows no trend, reduced seasonality, stable variance, and mean near zero. Outliers remain but autocorrelation is minimal, confirming the series is now stationary and ready for modeling.

5. Finding optimal classical model for forecasting

1. We will perform ACF and PACF test to identify the ideal model only for **1st Order differenced data**.

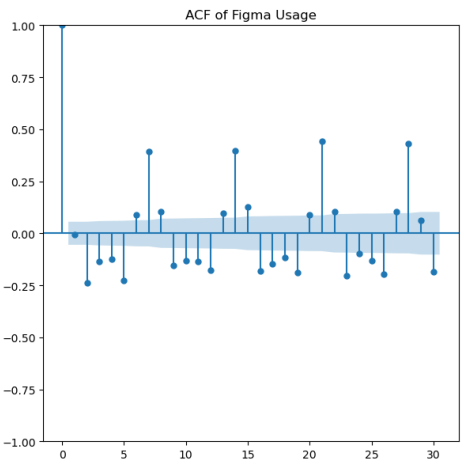


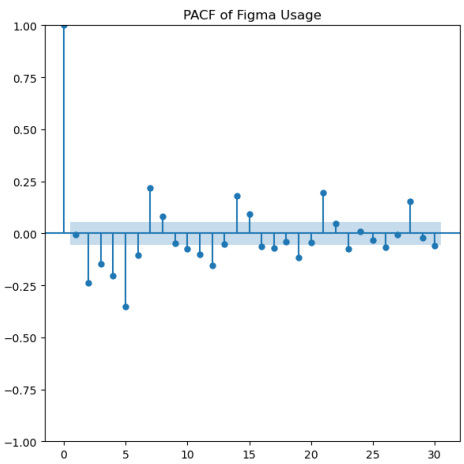


A. ACF: Sharp drop after lag 1 and mostly within confidence bounds — suggests a short-term MA(1) component.

B. PACF: Significant spike at lag 1 and tapering afterward — suggests a possible AR(1) process.
CONCLUSION-> The ACF and PACF patterns suggest an ARIMA model with seasonal effect indicating usage of SARIMA.

2. We will perform ACF and PACF test to identify the ideal model where we have **applied Box-Cox transformation to handle variance and then 1st Order differencing to make trend stationary**. We will choose between AR, MA, ARMA, ARIMA and SARIMA models to best fit our case.





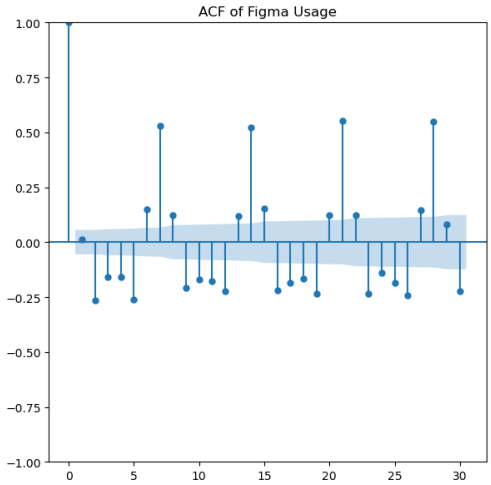
Conclusion-> The combination of: ACF: Sharp drop after lag 1 (→ MA(1)),

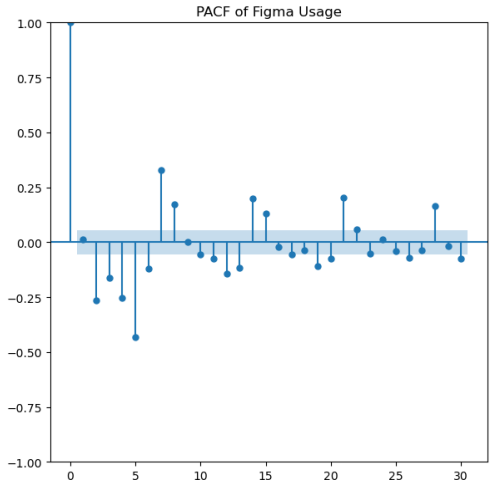
PACF: Strong spike at lag 1 then tapering (→ AR(1)),

suggests the time series is well-suited to an ARIMA(1,1,1) model (considering first-order differencing already applied). However, given earlier findings of weekly seasonality, a SARIMA model incorporating seasonal components (P,D,Q,s) is more appropriate.
Thus, a SARIMA(p=1, d=1, q=1)(P,D,Q,7) structure is a recommended starting point for capturing both short-term and seasonal dependencies.

3. In downstream analysis, we have observed a heavy impact of outliers affecting the classical models. Which we will discuss in the next section.

   Therefore, we will **winsorize outlier clipping along with Box-Cox transformation to handle variance and 1st Order differencing.** Winsorize is a bit less harsh than 1.5*IQR when it comes to outlier handling.





Conclusion->The updated ACF and PACF plots, after Winsorizing the tails, show: 1. ACF:
Still a strong spike at lag 1, but with more structured behavior at higher lags. Several significant spikes at lags 7, 14, and 21 suggest weekly seasonality (7-day cycle).
The pattern supports a short-term MA component and seasonal MA structure.
2. PACF:
A dominant spike at lag 1 followed by sharp tapering. Some minor spikes at seasonal lags (7, 14), indicating possible seasonal AR terms. This refined structure points toward an AR(1) component with seasonal AR terms.

6. Creating the model and initial analysis

Before applying modelling, we **first create a AIC/BIC functions for finding ideal NON-SEASONAL parameters** given AR, MA, ARMA/ARIMA (we will not focus on ARIMA because we have already done 1st order differencing so there will be no difference between ARMA and ARIMA) which depends on (P,d,Q) -> indicates the AutoRegression parameter i.e. the number of lagged observations in order to predict, d-> level of differencing (0 in our case) and Q->MA parameter, i.e. the number of lagged forecast errors used in the prediction. **Usually the lower the AIC/BIC value of (P,Q) the better the model**. We then **create a function to identify seasonal Orders using grid-**

search for SARIMA given Non-seasonal Order using AIC criterion. Lower the better.

Post this, we write our model functions(AR, MA, ARMA, ARIMA, and SARIMA) for training and forecasting.

We have created functions to call it multiple times after changing parameters.

A. CASE 1.

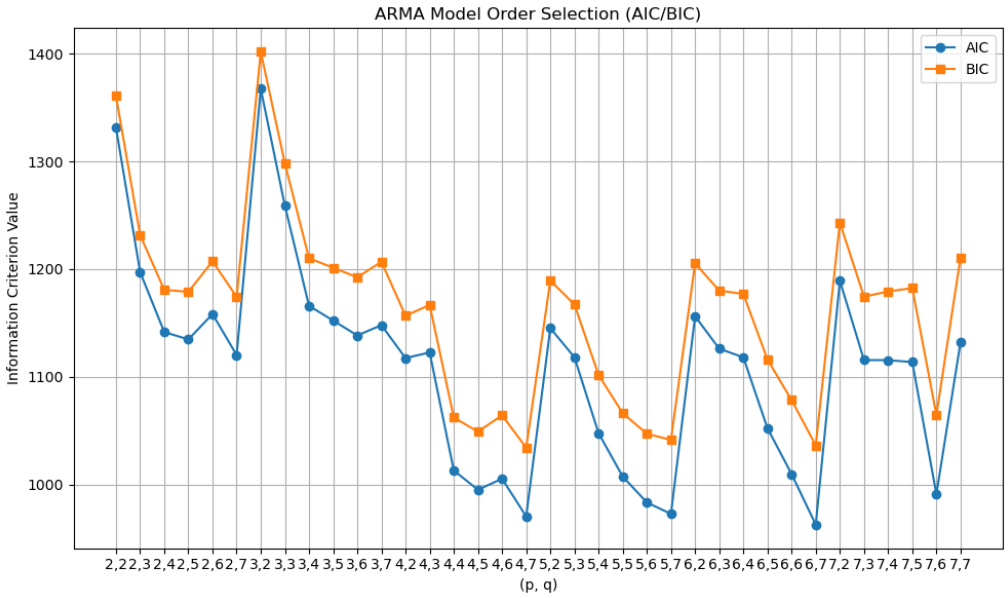
Here we have first applied Boxcox transform and then performed 1st order differencing. For model training we have used train-test split at 80-20.

We first perform initial analysis for AR, MA, ARMA and ARIMA.

CONCLUSION-> Given the **MSE value and forecast trend, spike, and seasonality, we have observed ARMA (with 1st order diff) or ARIMA with 0 differencing to be the best**, given the weekly seasonal nature our main focus will be on Seasonal ARIMA/SARMIA.

Only working on ARMA/ARIMA and SARIMA-BEST MODELS

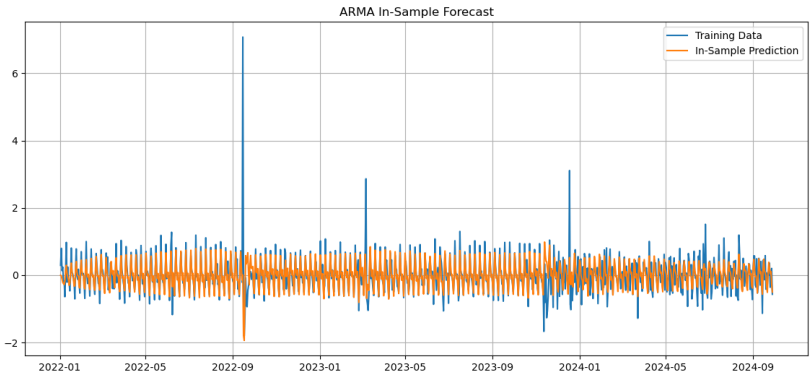
Identifying ideal seasonal and non-seasonal parametes.



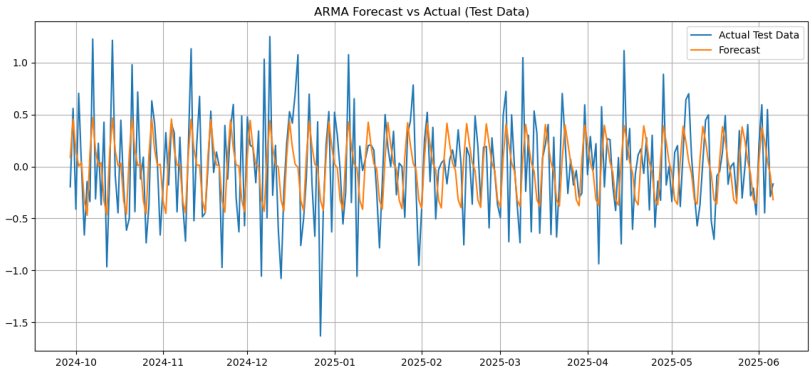
As per the graph-> Best pick (4,0,7)->highest AIC or (5,0,5)-> more robustness/generalized (balanced AIC/BIC) or (4, 6)->More stable and conservative.

As per Grid-search for seasonal order given non-seasonal orders. Our model parameters are **A. (4,0,7),(3, 0, 0, 7) OR (5,0,5), (3, 0, 3, 7) OR (4,0,6),(3, 0, 2, 7).**

For model with lowest MSE score:



ARMA Train Mean Squared Error: 0.15

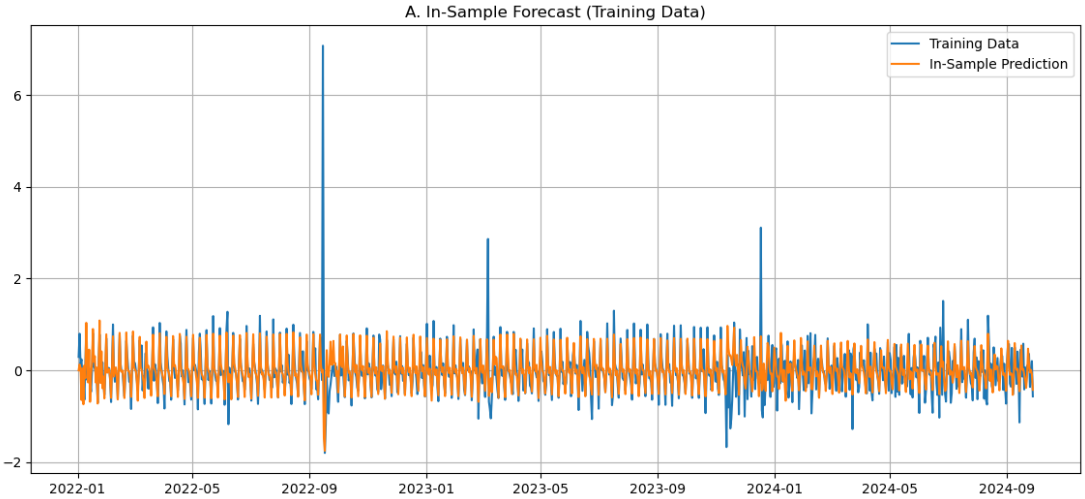


ARMA Test Data Mean Squared Error: 0.19

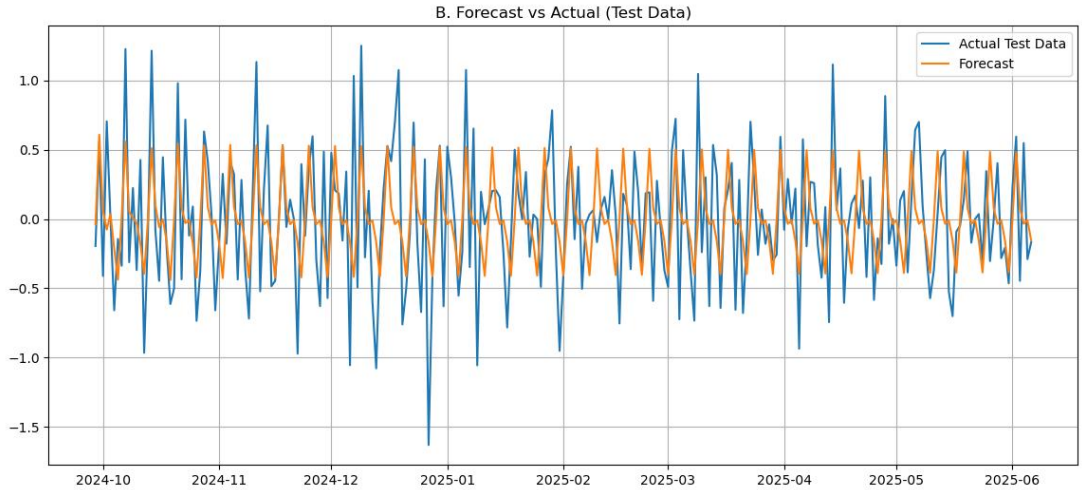
Conclusion->Despite low MSE values we observe that the forecast is not able to fully capture the seasonality and spikes.

MSE for 4,7-> (ARMA Train Mean Squared Error: 0.15, ARMA Test Data Mean Squared Error: 0.19)
MSE for 5,5-> (ARMA Train Mean Squared Error: 0.16, ARMA Test Data Mean Squared Error: 0.19)
MSE for 4,6-> (ARMA Train Mean Squared Error: 0.16, ARMA Test Data Mean Squared Error: 0.19)

SARIMAX model:



Train Mean Squared Error: 0.14



Test Data Mean Squared Error: 0.19

Conclusion->We observe that the model has low MSE values and is able to capture the seasonality. A high degree of variance is observed which needs trimming.

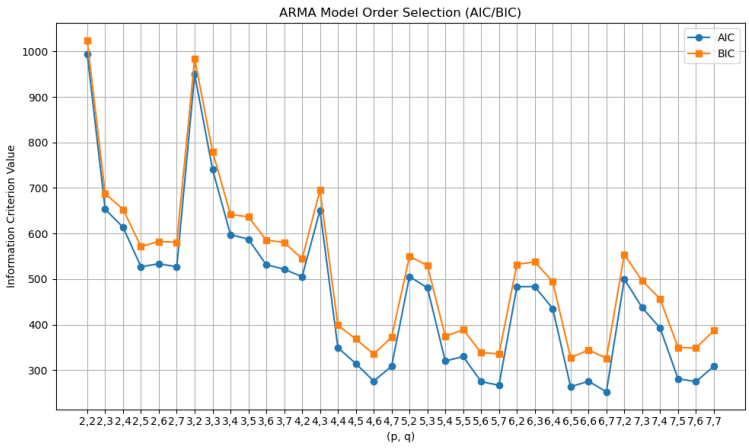
MSE for (4,0,7),(3, 0, 0, 7)-> (Train Mean Squared Error: 0.17, Test Data Mean Squared Error: 0.20).
MSE for (5,0,5), (3, 0, 3, 7) -> (Train Mean Squared Error: 0.14, Test Data Mean Squared Error: 0.19).
MSE for (4,0,6), (3, 0, 2, 7) -> (Train Mean Squared Error: 0.14, Test Data Mean Squared Error: 0.19).

B.CASE 2.

Along with Boxcox transform and 1st order differencing, we have also Winsorized tails. We did not perform this at first as transforming outliers might impact the model in terms of trend, seasonality.

For model training we have used train-test split at 80-20.

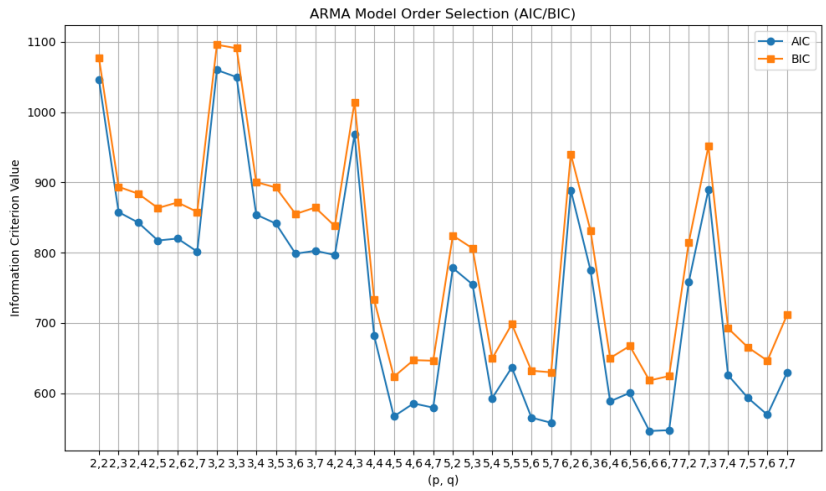
We have also created a 100-0 split to find the parameters for entire trained data.
However, this method later backfired because while training on the entire dataset we realized that the data post dip is mostly noise affecting the MSE score drastically. Therefore at a later stage we will perform cross-validation sampling where we vary the training size to find ideal model.



We will consider (6,0,7),(4,0,6),(5,0,7) and also (6,0,0) and based on grid search for non-seasonal orders, our models are(6,0,0),(1, 0, 1, 7), (4,0,7),(3, 0, 0, 7),(5,0,5),(1, 0, 1, 7), (4,0,6),(1, 0, 1, 7).

For high spikes, we have also included (6,0,0) to remove effect of MA.

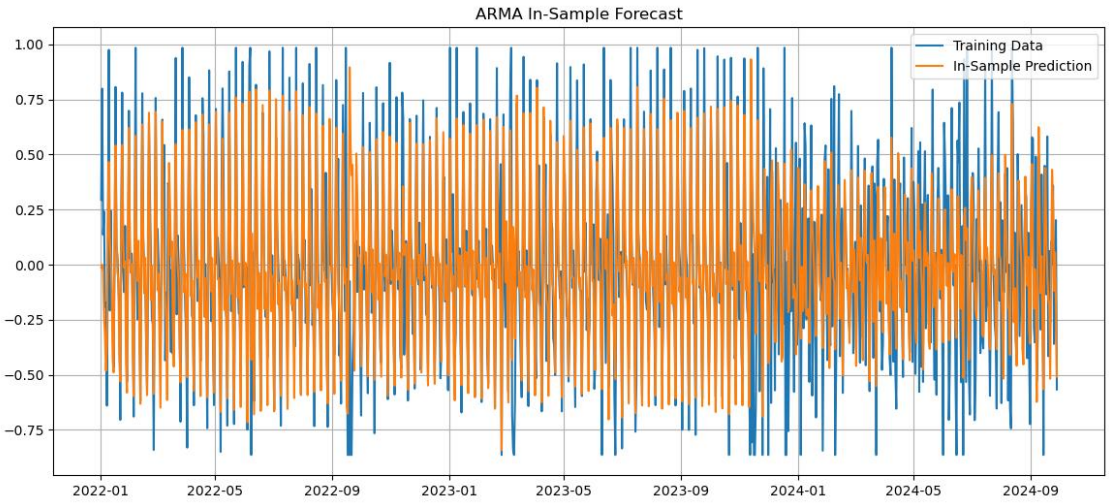
MODEL PARAMETERS CONSIDERING ENTIRE TRAINING DATA



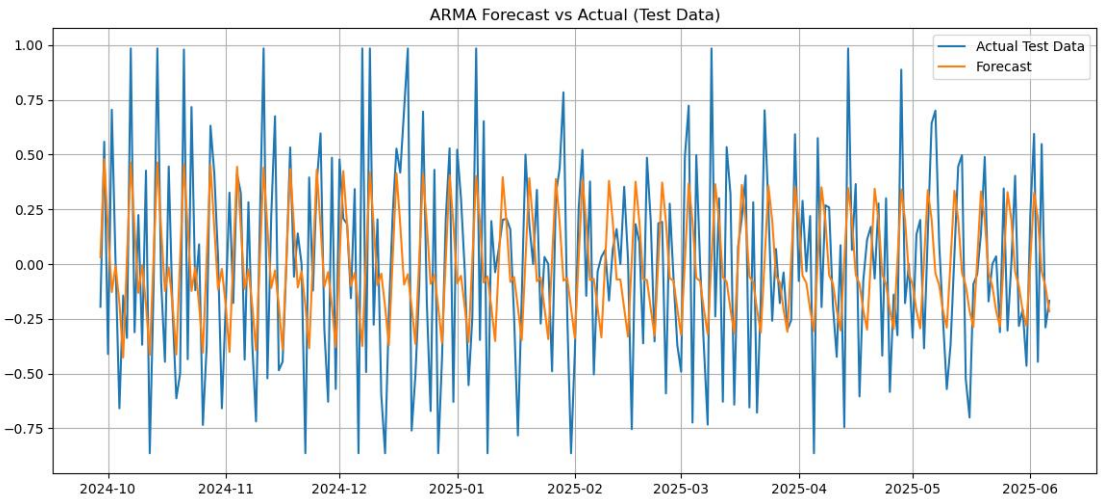
Our models-> (4,0,5),(3, 0, 2, 7)
(6,0,7), (0, 0, 0, 7)
(5,0,7), (0, 0, 0, 7)
(6,0,0), (1, 0, 1, 7)

Parameters training 80% of data differs from 100% data. We see this change because of increased downstream noise.

ARMA model post winsorizing outliers. (80-20 split)



ARMA Train Mean Squared Error: 0.07



ARMA Test Data Mean Squared Error: 0.17

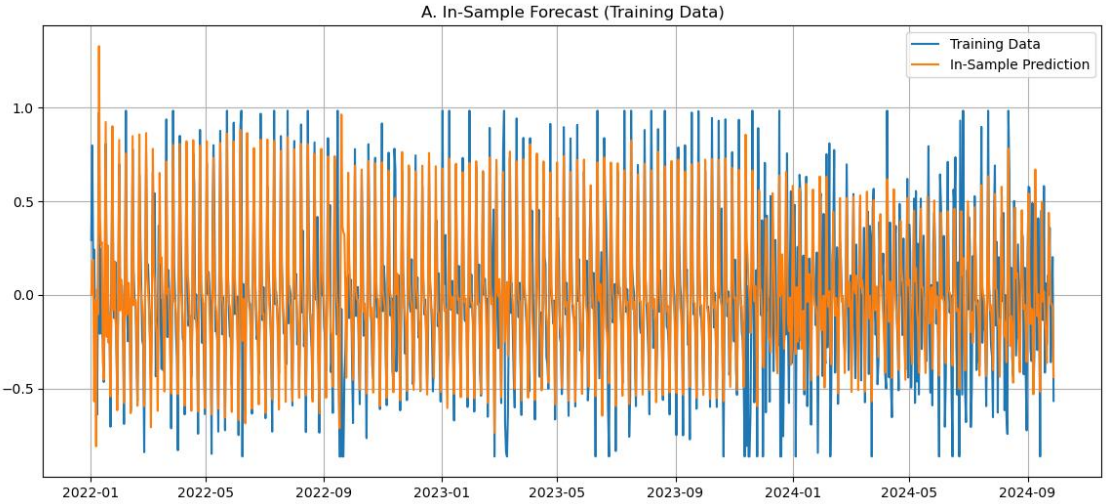
Conclusion-> Post treatment of outliers we have observed that the training MSE has gone down further and the variance has been somewhat captured. We'll look at SARIMA for capturing the trend better as there seem to be a lag.

(6,0,7)->ARMA Train Mean Squared Error: 0.07, ARMA Test Data Mean Squared Error: 0.17

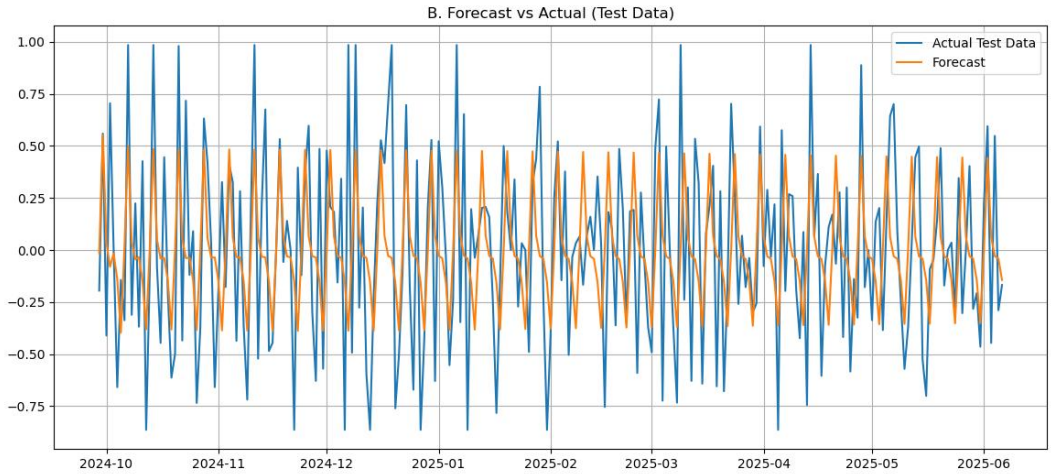
(4,0,6)-> ARMA Train Mean Squared Error: 0.08, ARMA Test Data Mean Squared Error: 0.17

(5,0,7)->ARMA Train Mean Squared Error: 0.07, ARMA Test Data Mean Squared Error: 0.17

SARIMA model post winsorizing outliers. (80-20 split)



Train Mean Squared Error: 0.07



Test Data Mean Squared Error: 0.17

Conclusion: Post treatment of outliers we have observed that the training MSE has gone down further and the variance has been somewhat captured. The trend seems to have been captured. However despite all the transformations it seems the model is unable to fit the constant spikes properly.

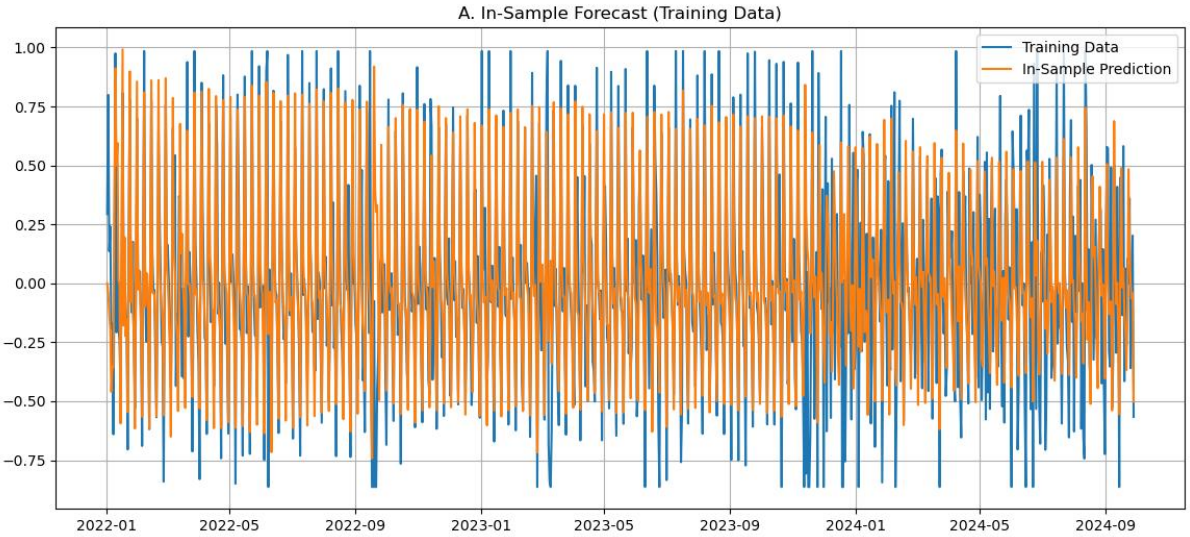
(4,0,7),(3, 0, 0, 7)->SARIMA Train Mean Squared Error: 0.08, Test Data Mean Squared Error: 0.20

(5,0,5),(1, 0, 1, 7)->SARIMA Train Mean Squared Error: 0.07, Test Data Mean Squared Error: 0.17

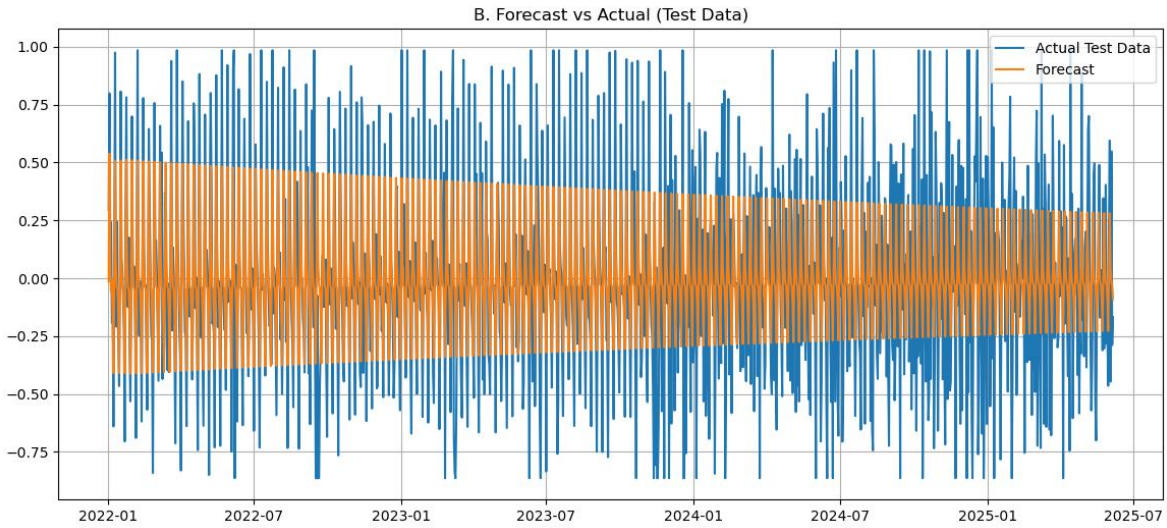
(4,0,6),(1, 0, 1, 7)->SARIMA Train Mean Squared Error: 0.07, Test Data Mean Squared Error: 0.17

(6,0,0), (1, 0, 1, 7)->SARIMA Train Mean Squared Error: 0.07, Test Data Mean Squared Error: 0.17 **(This has performed well surprisingly)**

After applying training on 80% data and testing on the entire dataset.

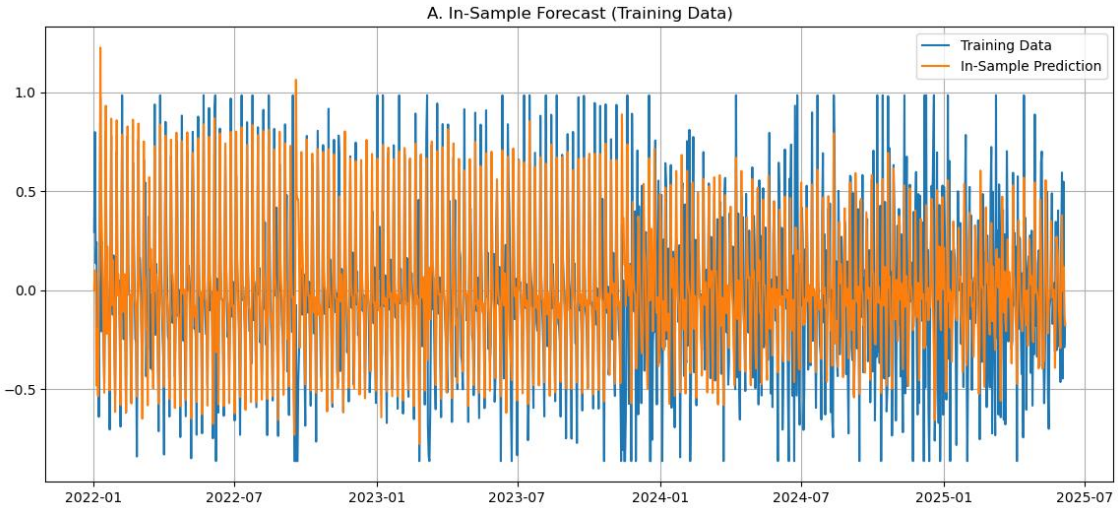


Train Mean Squared Error: 0.07

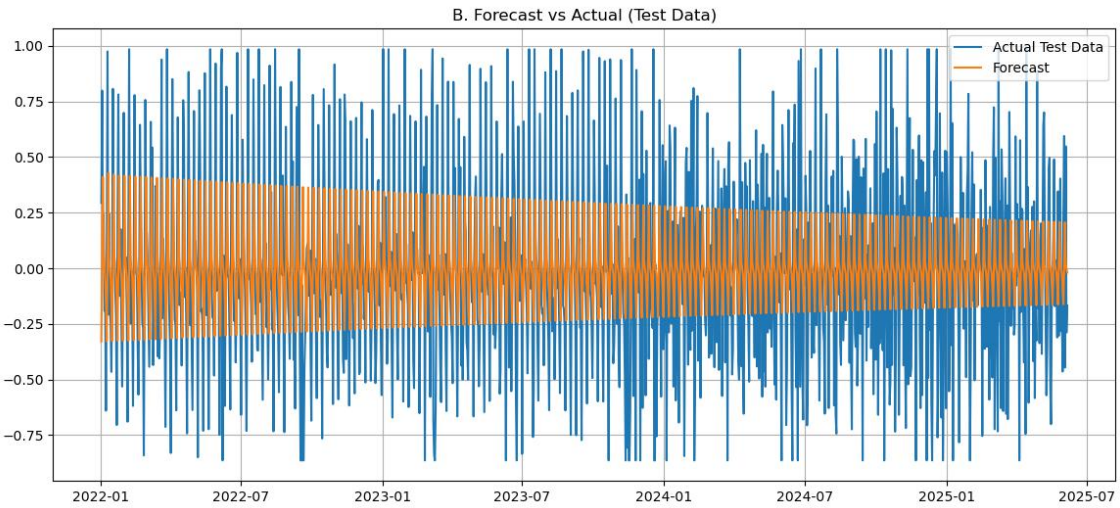


Test Data Mean Squared Error: 0.11

VS applying training and testing on the entire dataset. (keeping same parameters)



Train Mean Squared Error: 0.08



Test Data Mean Squared Error: 0.19

Observation-> We have observed that the model learns decently at 80% training data(MSE->0.11) as compared to 100% train data(MSE->0.19).

Reason-> This is mainly due to the structure of page view analysis. The model perfectly about the trend, and seasonality before the spike, post the spike and before the dip, the model tends to learn a less due to increased noise during this phase.

Finally, the model does not learn much beyond the dip because of no trend, slight seasonality, and mostly noise.

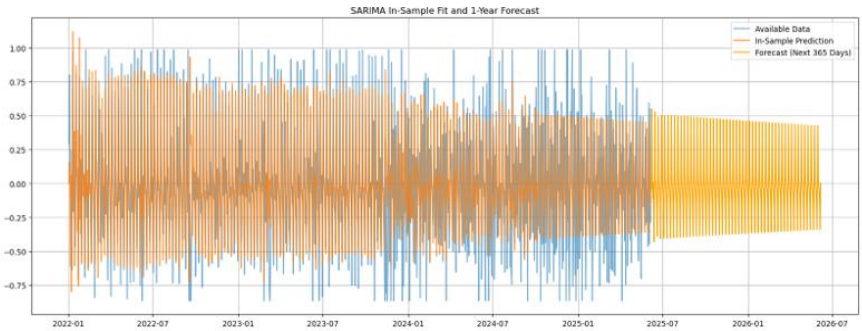
Therefore, we will first vary the training data 0.7,0.75, 0.8,0.85,0.9,0.95 and perform testing over the entire data, calculate MSE, perform inverse transform (This Reverses the effect of box-cox transform, then cumulative sums the differences to finally plot the data against the original) and observe how the predictions show trend, seasonality and noise.

Conclusion-> Observation seasonal_order (1,0,1,7) massively shoots up the predcition post a spike however this in combination with high AR (6,0,0) gives decent perdiction.

(4,0,6) is observed to be the most stable NON-SEASONAL Order...

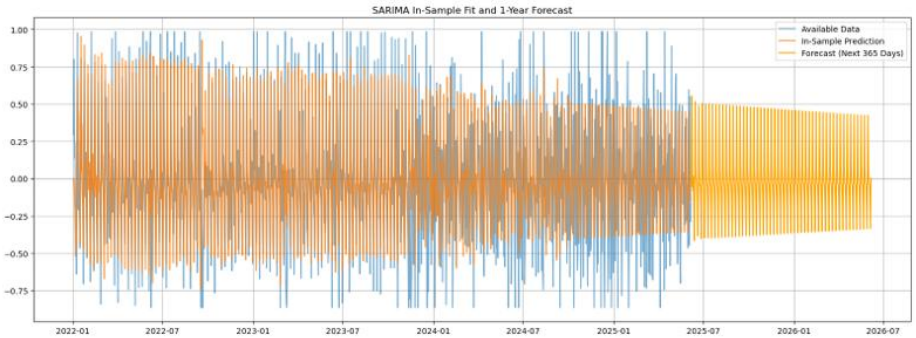
We have observed the model to learn most at 80% of the entire data, which mostly skips the post-dip phase because of high noise.

7. CONCLUSION



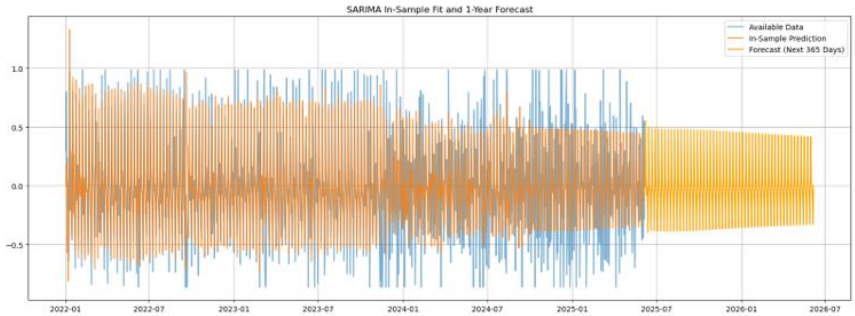
InSample model Mean Squared Error: 0.0910

Model 1 SARIMA, Parameters-> 0.8 ((4, 0, 6), (3, 0, 2, 7))



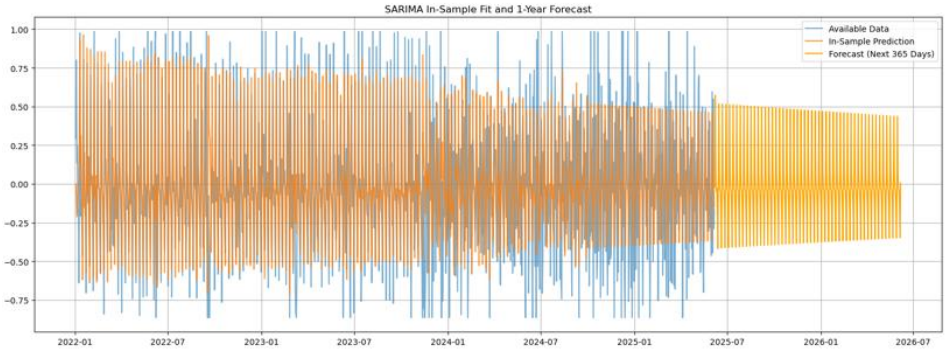
InSample model Mean Squared Error: 0.0896

Model 2 SARIMA, Parameters-> 0.8 ((5, 0, 5), (3, 0, 2, 7))



InSample model Mean Squared Error: 0.0902

MODEL 3 SARIMA, Parameters-> 0.8 ((4, 0, 6), (1, 0, 1, 7))

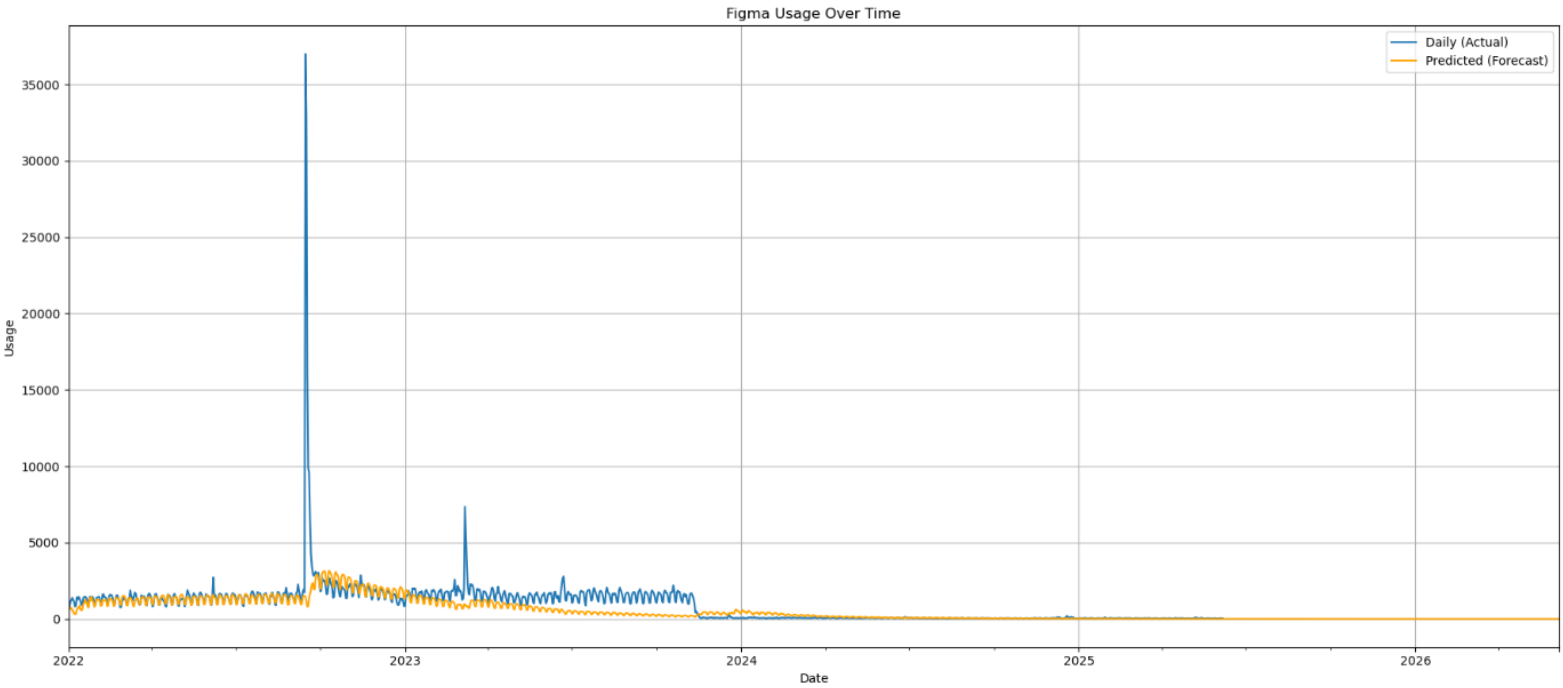


InSample model Mean Squared Error: 0.0899

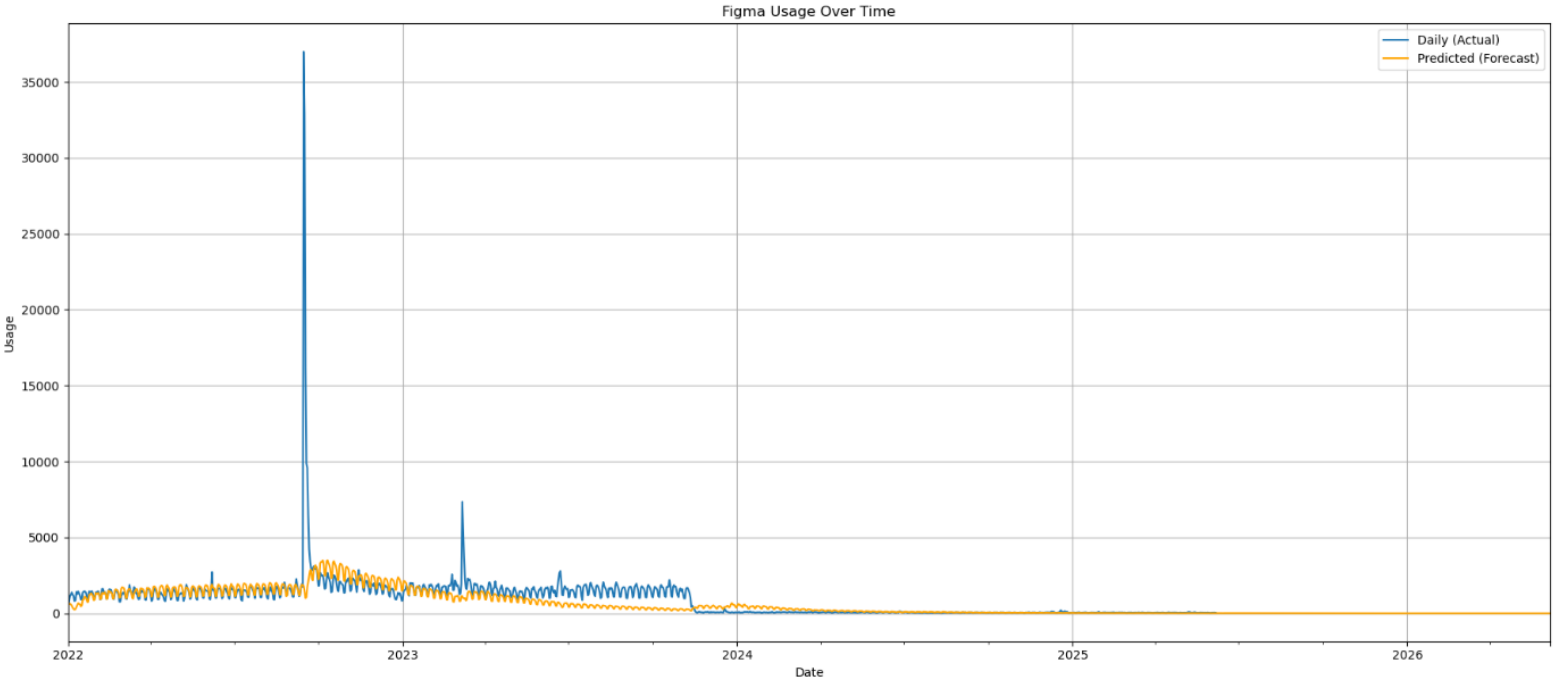
MODEL 4 SARIMA, Parameters-> 0.8 ((6, 0, 0), (1, 0, 1, 7))

Conclusion-> For all the 4 final models, we have observed similar score for InSample MSE.

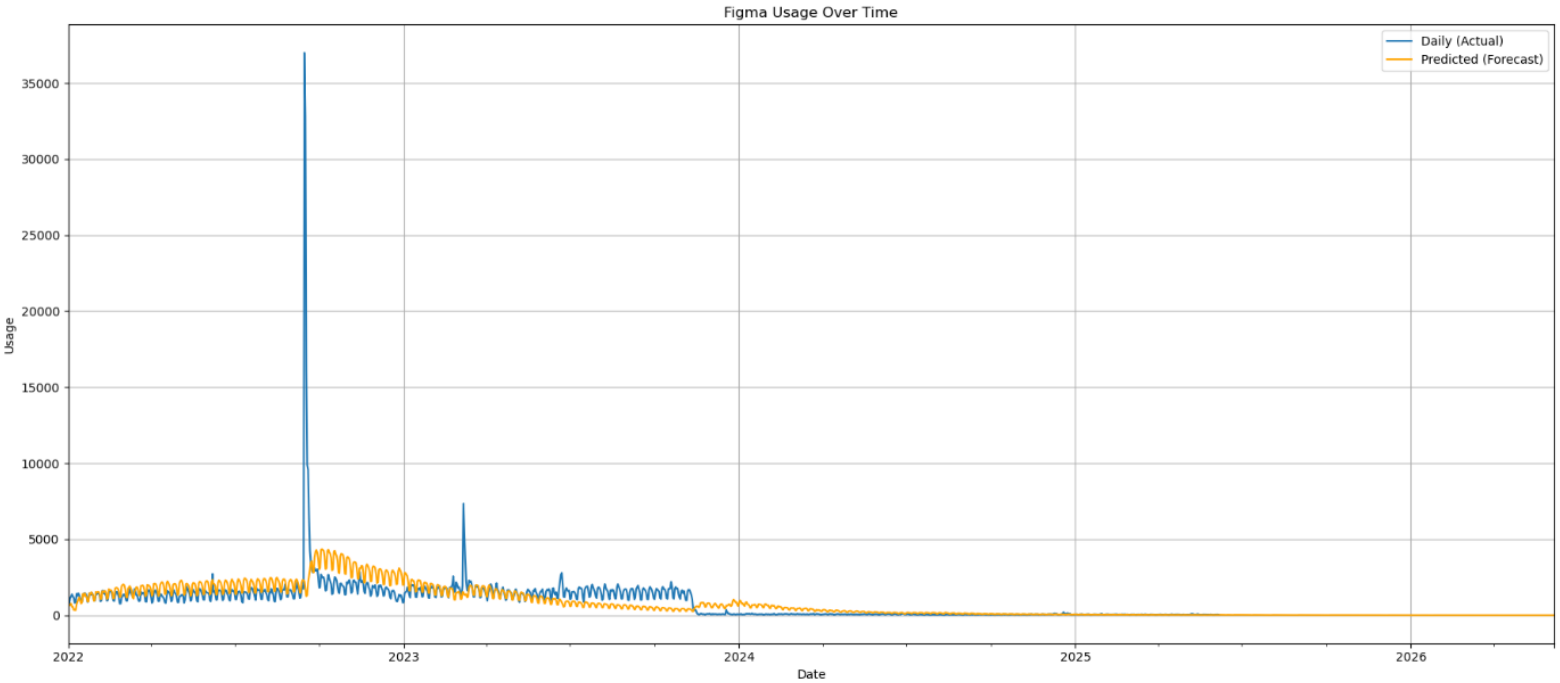
Performing inverse transform and forecasting the data...



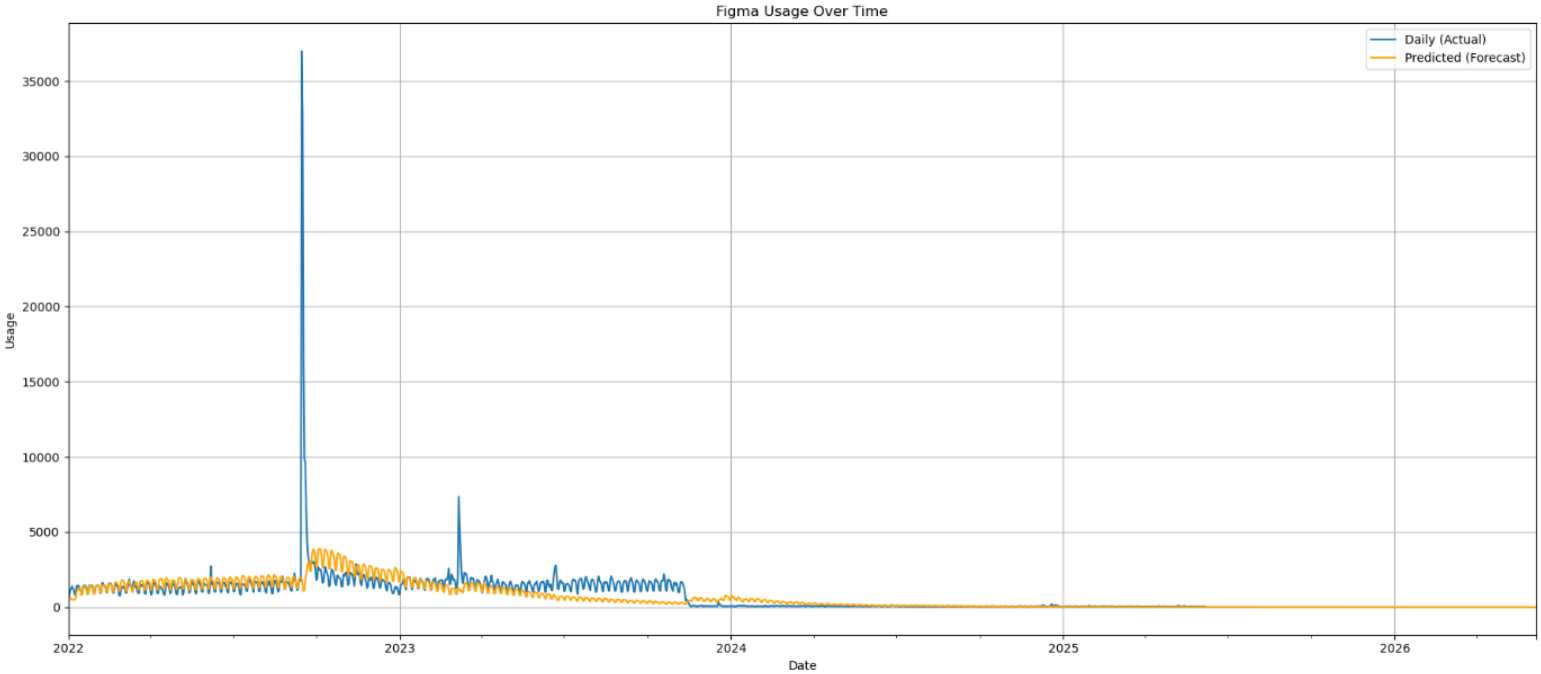
1. **Model 1 SARIMA, Parameters-> 0.8 ((4, 0, 6), (3, 0, 2, 7)) -> Stable + seasonality, decent spike handling**



2. **MODEL 2 SARIMA, Parameters-> 0.8 ((5, 0, 5), (3, 0, 2, 7)) -> Similar to Model 1, slightly higher AIC/BIC**



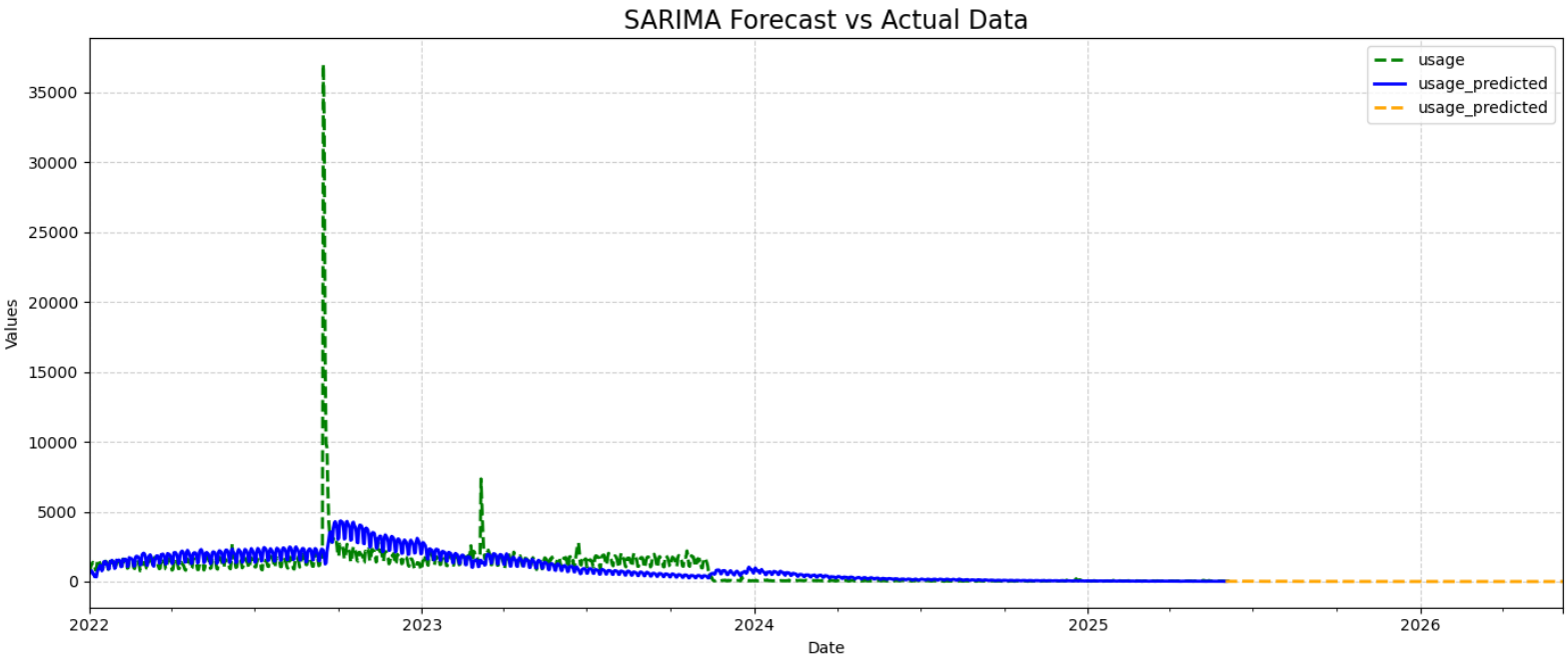
3. **MODEL 3 SARIMA, Parameters-> 0.8 ((4, 0, 6), (1, 0, 1, 7)) ->Stable, best generalizer**

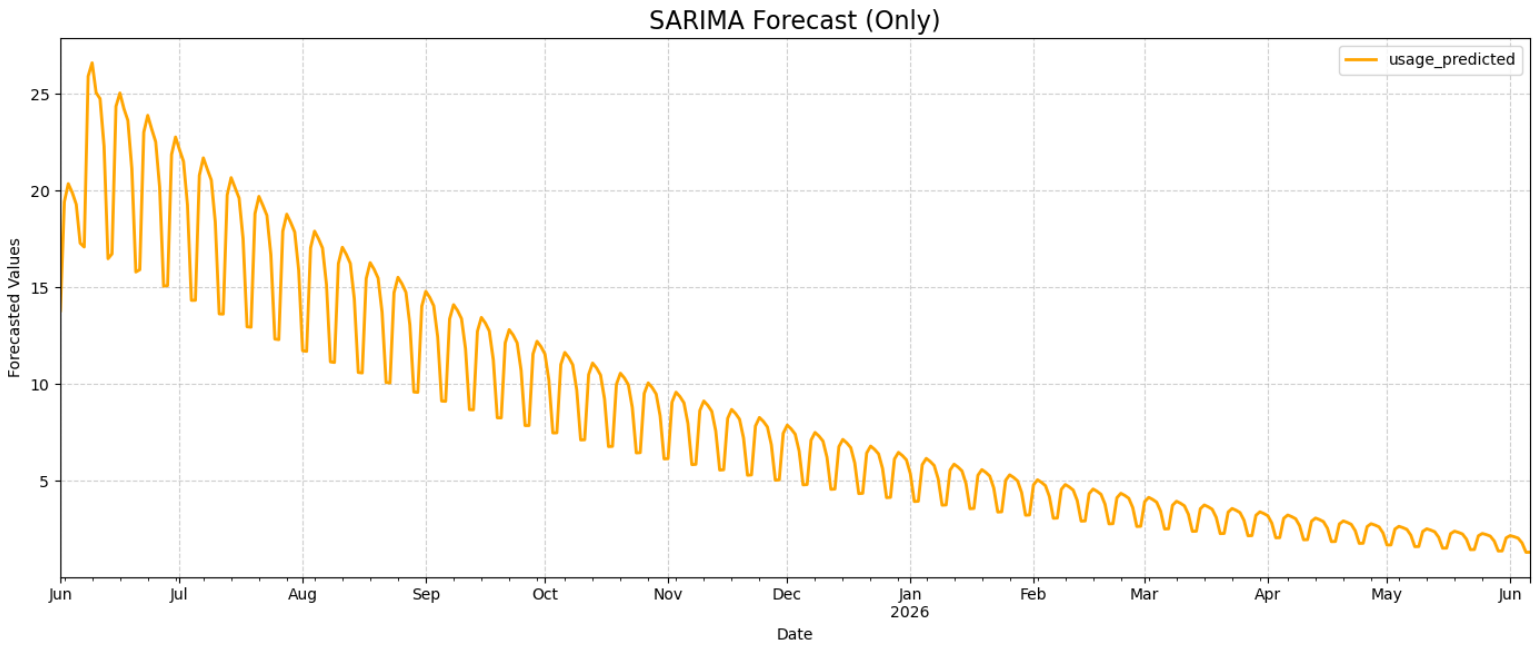


4. **MODEL 4 SARIMA, Parameters-> 0.8 ((6, 0, 0), (1, 0, 1, 7)) ->Best at post-spike prediction, weak MA.**

FINAL CONCLUSION:

SARIMA ((4, 0, 6), (1, 0, 1, 7)) emerges as the best overall model due to its optimal balance of stability, generalization, and effective seasonality capture with a simpler seasonal structure compared to **SARIMA ((4, 0, 6), (3, 0, 2, 7)) and SARIMA ((5, 0, 5), (3, 0, 2, 7))**. It avoids overfitting to short-term noise while maintaining reliable forecasts across different time periods, making it robust beyond specific events like usage spikes. Although **SARIMA ((6, 0, 0), (1, 0, 1, 7))** performs well in post-spike prediction, its lack of moving average components limits its broader applicability. Overall, **SARIMA ((4, 0, 6), (1, 0, 1, 7))** versatility and clean generalization make it the strongest and most dependable choice for long-term forecasting.





FORECAST CONCLUSION:

The SARIMA forecast for Figma pageviews over the next year shows a clear and steady decline in user activity, with consistent weekly seasonality. The model captures the cyclical pattern of usage—likely reflecting higher engagement on weekdays and lower on weekends—while projecting an overall downward trend. By mid-2026, the daily pageviews are expected to fall to very low levels, indicating a continued decrease in engagement if no changes occur.

This forecast suggests that the current decline in usage is not temporary or random, but part of a broader structural trend. The absence of any predicted recovery or spikes implies that, based on historical data, the system expects user interest to diminish further over time. While the SARIMA model handles seasonality and trend well, it does not account for external factors such as product updates, user acquisition efforts, or organizational shifts.

As a result, this projection should be seen as a baseline scenario under stable conditions. It is useful for identifying long-term risks in user engagement and can help guide intervention strategies. For more accurate forecasting in dynamic environments, integrating external variables or moving toward hybrid models may be required. Overall, the forecast provides a clear signal of declining user interest.

FURTHER IMPROVEMENTS IN MODEL:

These models are inherently limited in handling abrupt structural shifts, irregular anomalies, and nonlinear dependencies. To move beyond this ceiling, incorporating machine learning or hybrid models (e.g., SARIMA + XGBoost, LSTM, or Prophet with regressors) will be necessary to capture deeper patterns and improve predictive accuracy, especially in volatile or data-rich environments.